Лабораторная работа № 8

Тема: Управление процессами, системными службами и мониторинг многозадачности в Linux и Windows Server

Цель работы:

1. Научиться идентифицировать и анализировать ресурсы процессов (PID, CPU, RAM) в Linux и Windows Server.
2. Освоить управление жизненным циклом процессов: запуск в фоновом режиме, изменение приоритетов, отправка сигналов завершения.
3. Научиться диагностировать состояние системных служб и выявлять аномалии в многопоточных приложениях.

Используемое ПО: ВМ Linux (Debian/Ubuntu), ВМ Windows Server

ТЕОРЕТИЧЕСКИЙ ЧЕК-ЛИСТ

Перед выполнением работы важно знать:

- **Вытесняющая многозадачность:** Планировщик ОС сам решает, когда переключать задачи и распределять кванты процессорного времени.
- **Процесс vs Поток:** Процесс — это тяжелая структура со своим изолированным адресным пространством и PID; потоки — легкие единицы выполнения внутри процесса, разделяющие его ресурсы.

ЧАСТЬ 1. РАБОТА В ОС LINUX (Управление жизненным циклом и сигналами)

Исследование процессов, потоков, сигналов и системных служб.
Цель работы: Исследуйте процессы Linux и их основные характеристики. Научитесь определять PID и PPID, отслеживать состояние процесса, запускать задачи в фоне, изменять приоритет, отправлять сигналы, анализировать потоки, открытые файлы и сетевые соединения. Исследуйте связь между службой systemd и запущенным процессом, а также создайте безопасный учебный zombie-процесс.
Все эксперименты выполняйте только на учебной виртуальной машине.

Этап 1. Исследование текущего командного интерпретатора
Командный интерпретатор Bash, в котором вы сейчас работаете, тоже является обычным процессом Linux.
1. Получите PID текущего Bash
Выполните:
echo $$
Переменная:
$$
содержит PID текущего командного интерпретатора.
Например:
1245
Зафиксируйте PID.

2. Получите подробные сведения о процессе
Выполните:
ps -fp $$
Команда ps показывает сведения о процессах.
Параметры:

- -f — расширенный формат;
- -p — показать процесс с указанным PID;
- $$ — PID текущего Bash.

Пример:

```
UID        PID  PPID  C STIME TTY          TIME CMD
student   1245  1201  0 09:15 pts/0    00:00:00 bash
```

Основные поля:

- UID — пользователь, от имени которого работает процесс;
- PID — идентификатор процесса;
- PPID — идентификатор родительского процесса;
- STIME — время запуска;
- TTY — терминал;
- TIME — использованное процессорное время;
- CMD — выполняемая команда.

Зафиксируйте:

- пользователя;
- PID;
- PPID;
- выполняемую команду.

3. Исследуйте дерево процессов
Выполните:
pstree -p
Команда показывает процессы в виде дерева:

```
родитель
  └── дочерний процесс
```

Параметр:

-p
добавляет PID.
Найдите свой процесс bash.
Определите:

- какой процесс является его родительским;
- PID родительского процесса.

Этап 2. Запуск процесса в фоновом режиме
Запустите простой процесс:
sleep 300 &
Команда sleep 300 ничего не делает в течение 300 секунд.
Символ:
&
запускает процесс **в фоне**, поэтому терминал остаётся доступным для дальнейшей работы.
После запуска оболочка выведет примерно:
[1] 2314
Где:

- [1] — номер фоновой задачи;
- 2314 — PID.

1. Сохраните PID
Выполните:
PID=$!
Переменная:
$!
содержит PID последнего фонового процесса.
Проверьте:
echo $PID
Зафиксируйте PID процесса.

2. Получите список фоновых задач
Выполните:
jobs
Команда jobs показывает фоновые задачи, запущенные из текущего терминала.
Пример:
[1]+ Running sleep 300 &
Убедитесь, что процесс работает.

Этап 3. Исследование состояния процесса
Получите сведения:
ps -o pid,ppid,stat,ni,cmd -p $PID
Команда выводит только выбранные поля:

- pid — PID;

- ppid — родительский PID;
- stat — состояние;
- ni — приоритет Nice;
- cmd — команда.

Пример:
PID PPID STAT NI CMD
2314 1245 S   0 sleep 300
Основные состояния:
R → Running, выполняется
S → Sleeping, ожидает
T → Stopped, остановлен
Z → Zombie
Для sleep вы, скорее всего, увидите:
S
Потому что процесс большую часть времени ожидает.
Зафиксируйте текущее состояние.

Этап 4. Остановка и продолжение процесса
Linux позволяет отправлять процессам специальные сигналы. Сигнал — это команда ядру выполнить определённое действие с процессом.
1. Приостановите процесс
Выполните:
kill -STOP $PID
Несмотря на название kill, команда может не только завершать процессы, но и отправлять различные сигналы.
SIGSTOP приостанавливает выполнение процесса.

2. Проверьте состояние
Выполните:
ps -o pid,stat,cmd -p $PID
Теперь в STAT должна появиться буква: T
Она означает:
процесс остановлен.
Зафиксируйте состояние.

3. Продолжите выполнение
Выполните:
kill -CONT $PID
Сигнал SIGCONT разрешает остановленному процессу продолжить работу.
Снова:
ps -o pid,stat,cmd -p $PID
Убедитесь, что состояние T исчезло.

Этап 5. Корректное завершение процесса
Отправьте:
kill -TERM $PID
SIGTERM — стандартный сигнал с просьбой корректно завершить работу.
Процесс получает возможность:

- сохранить данные;
- закрыть файлы;
- освободить ресурсы;
- нормально завершиться.

Проверьте:
ps -p $PID
Если процесс завершён, его строка больше не появится.
Зафиксируйте результат.

Этап 6. SIGTERM и SIGKILL
Создайте ещё один процесс:
sleep 300 &
PID=$!
Проверьте:
ps -p $PID
Теперь принудительно завершите:
kill -KILL $PID
или числовой вариант:
kill -9 $PID
SIGKILL немедленно завершает процесс.
Процесс не может проигнорировать этот сигнал и не получает возможности корректно завершить работу.
Проверьте:
ps -p $PID
Ответьте:
Почему администратору обычно лучше сначала использовать SIGTERM, а не SIGKILL?

Этап 7. Создание CPU-нагрузки
Теперь запустите процесс, который будет активно использовать процессор.
Выполните:
yes > /dev/null &
Команда yes непрерывно генерирует строки.
Конструкция:
> /dev/null
отправляет вывод в специальное устройство /dev/null, то есть данные просто отбрасываются.
В результате процесс активно работает, но не заполняет экран.

Сохраните PID:
PID=$!

1. Получите сведения о нагрузке
Выполните:
ps -o pid,ni,%cpu,%mem,cmd -p $PID
Поля:

- NI — значение Nice;
- %CPU — использование процессора;
- %MEM — использование RAM.

Зафиксируйте:

- PID;
- Nice;
- %CPU;
- %MEM.

Этап 8. Мониторинг процесса через top
Выполните:
top
top показывает процессы и ресурсы системы в реальном времени.
Найдите процесс:
yes
Обратите внимание на:

- PID;
- %CPU;
- %MEM;
- NI;
- состояние.

Для выхода нажмите: q
Определите, занимает ли процесс значительную часть одного CPU.

Этап 9. Изменение приоритета процесса
Linux позволяет изменять относительный приоритет процесса с помощью Nice.
Чем **больше Nice**, тем процесс «вежливее» по отношению к другим процессам и тем ниже его приоритет.
Выполните:
renice -n 15 -p $PID
Команда изменяет Nice процесса на: 15
Проверьте:
ps -o pid,ni,%cpu,cmd -p $PID
Зафиксируйте новое значение NI.
Значение Nice само по себе не гарантирует мгновенного падения %CPU. Оно влияет на то, как планировщик распределяет процессорное время при конкуренции между процессами.

После эксперимента завершите:
kill $PID

Этап 10. Исследование процессов по потреблению ресурсов
Получите процессы, использующие больше всего CPU:
ps aux --sort=-%cpu | head
Параметр:
--sort=-%cpu
сортирует процессы от большего использования CPU к меньшему.
head оставляет первые строки.
Определите три процесса с наибольшим использованием CPU.
Затем:
ps aux --sort=-%mem | head
Эта команда сортирует процессы по использованию оперативной памяти.
Определите три процесса с наибольшим использованием RAM.
Сравните результаты.
Ответьте:
Всегда ли процесс, который больше всего использует CPU, одновременно использует больше всего RAM?

Этап 11. Исследование потоков процесса
Один процесс может содержать несколько потоков. Эта тема отдельно рассматривается в лекции.
Выберите запущенный системный процесс.
Например, найдите SSH:
pgrep sshd
Если SSH отсутствует, выберите другой процесс.
Получите PID и выполните:
ps -T -p PID
Например:
ps -T -p 765
Параметр:
-T
показывает потоки внутри процесса.
В выводе могут присутствовать:

- PID — идентификатор процесса;
- SPID — идентификатор потока;
- CMD — команда.

Определите:

- сколько потоков содержит выбранный процесс;
- одинаковый ли PID у строк;
- отличаются ли идентификаторы потоков.

Этап 12. Исследование открытых файлов процесса
Процесс может работать с файлами, терминалами, библиотеками и другими объектами. В лекции открытые файлы относятся к ресурсам процесса.
Создайте процесс:
sleep 300 &
PID=$!
Посмотрите его файловые дескрипторы:
ls -l /proc/$PID/fd
Каталог:
/proc/PID/fd
содержит открытые файловые дескрипторы процесса.
Обычно будут присутствовать:
0
1
2
Это стандартные дескрипторы:
0 → stdin — стандартный ввод
1 → stdout — стандартный вывод
2 → stderr — стандартный вывод ошибок
Зафиксируйте, какие дескрипторы открыты.
После исследования:
kill $PID

Этап 13. Исследование сетевых процессов
Получите список TCP-портов:
ss -lntp
Параметры:

- -l — только listening sockets;
- -n — показывать номера портов, а не имена;
- -t — TCP;
- -p — показать процесс.

Если работает SSH, найдите строку с портом:
22
Например:
LISTEN 0 128 0.0.0.0:22 ...
Определите:

- локальный IP;
- порт;
- имя процесса;
- PID процесса.

Получите PID SSH отдельно:
pgrep sshd

Этап 14. Связь службы с процессом
Служба systemd — это объект управления, а реальную работу выполняет один или несколько процессов.
Исследуйте SSH:
systemctl status ssh
Если SSH не установлен, используйте:
systemctl status cron
Обратите внимание на состояние:
active (running)
Получите MainPID:
systemctl show ssh -p MainPID
Например:
MainPID=765
Теперь:
ps -fp 765
Зафиксируйте:
Служба:
PID:
Процесс:
Пользователь:
Составьте цепочку:
ssh.service
↓
MainPID
↓
sshd

Этап 15. Исследование состояния служб
Получите список работающих служб:
systemctl list-units --type=service --state=running
Команда показывает только службы, которые работают сейчас.
Зафиксируйте любые пять служб.

Получите список неисправных units:
systemctl --failed
Если проблем нет, вывод может быть пустым.
Определите, имеются ли failed units.

Теперь выполните:
systemctl list-unit-files --type=service
Эта команда показывает уже не только работающие службы, а установленные unit-файлы и их настройки автозапуска.
Например:
ssh.service enabled
cups.service enabled

some.service disabled
Основные состояния:
enabled → запускается автоматически
disabled → автоматически не запускается
Найдите ssh.service или cron.service и зафиксируйте его состояние.

Этап 16. Создание учебного zombie-процесса
Zombie-процесс — это уже завершившийся дочерний процесс, информация о котором ещё остаётся в таблице процессов, потому что родительский процесс не получил его статус завершения.
На этом этапе сравните две версии программы:

- в первой версии zombie **появляется**;
- во второй версии родитель корректно получает статус дочернего процесса, поэтому zombie **не остаётся**.

1. Создайте первую версию программы
Создайте файл:
nano zombie_bad.py
Введите следующий код:

```
#!/usr/bin/env python3

import os
import time

# Создаём дочерний процесс
pid = os.fork()

if pid == 0:
    # Эта часть выполняется дочерним процессом
    print("Child PID:", os.getpid())

    # Дочерний процесс сразу завершается
    os._exit(0)

else:
    # Эта часть выполняется родительским процессом
    print("Parent PID:", os.getpid())
    print("Child PID:", pid)

    # Родитель НЕ получает статус завершения дочернего процесса.
    # Пока родитель спит, завершившийся дочерний процесс
    # остаётся в таблице процессов как Zombie.
    time.sleep(60)
```

В питоне отступы имеют значение, код должен выглядеть так:

```
  GNU nano 8.4                          zomby_bad.py *
#!/usr/bin/env python3

import os
import time

# Создаём дочерний процесс
pid = os.fork()

if pid == 0:
    # Эта часть выполняется дочерним процессом
    print("Child PID:", os.getpid())

    # Дочерний процесс сразу завершается
    os._exit(0)

else:
    # Эта часть выполняется родительским процессом
    print("Parent PID:", os.getpid())
    print("Child PID:", pid)

    # Родитель НЕ получает статус завершения дочернего процесса.
    # Пока родитель спит, завершившийся дочерний процесс
    # остаётся в таблице процессов как Zombie.
    time.sleep(60)
```

Сохраните файл.
Сделайте его исполняемым:
chmod +x zombie_bad.py
Запустите:
./zombie_bad.py &
Или:
python3 zombie_bad.py &

Программа выведет примерно:
Parent PID: 2450
Child PID: 2451
Здесь:

- 2450 — PID родительского процесса;
- 2451 — PID дочернего процесса.

2. Найдите zombie-процесс
Пока родительский процесс находится в sleep(60), выполните:
ps -eo pid,ppid,stat,cmd | grep Z
Найдите строку с состоянием:
Z
Например:
2451 2450 Z [python3] <defunct>
Пояснение:

- PID — PID zombie-процесса;
- PPID — PID его родителя;

- STAT=Z — состояние Zombie;
- <defunct> — процесс уже завершён.

Зафиксируйте:

- PID zombie;
- PPID;
- значение STAT.

3. Объясните, почему появился zombie

Логика первой программы:

Родитель
↓
создаёт дочерний процесс
↓
дочерний процесс завершается
↓
родитель НЕ вызывает wait()
↓
статус завершения не получен
↓
Zombie

Через 60 секунд родительский процесс тоже завершится, после чего zombie должен исчезнуть.

Повторите:

```
ps -eo pid,ppid,stat,cmd | grep Z
```

Определите, исчез ли созданный zombie.

4. Создайте вторую версию программы — без zombie

Теперь создайте файл:

```
nano zombie_good.py
```

Введите:

```
#!/usr/bin/env python3

import os
import time

# Создаём дочерний процесс
pid = os.fork()

if pid == 0:
    # Эта часть выполняется дочерним процессом
    print("Child PID:", os.getpid())

    # Дочерний процесс завершает работу
```

```
    os._exit(0)

else:
    # Эта часть выполняется родительским процессом
    print("Parent PID:", os.getpid())
    print("Child PID:", pid)

    # Родитель ждёт завершения именно этого дочернего процесса.
    # После завершения дочернего процесса родитель получает
    # его статус и удаляет запись из таблицы процессов.
    os.waitpid(pid, 0)

    print("Child process finished.")
    print("Zombie не остаётся.")

    # Родитель продолжает работать ещё 60 секунд,
    # чтобы можно было проверить таблицу процессов.
    time.sleep(60)
```

Сделайте файл исполняемым:
`chmod +x zombie_good.py`
Запустите:
`./zombie_good.py &`

5. Проверьте, появился ли zombie
Сразу после запуска выполните:
`ps -eo pid,ppid,stat,cmd | grep Z`
Определите, появился ли новый zombie-процесс от zombie_good.py.
В нормальной ситуации его не должно быть.
Причина в строке:
os.waitpid(pid, 0)
Эта команда означает:
Родительский процесс ждёт завершения дочернего процесса и получает его статус завершения.
После этого ядру больше не нужно хранить запись о завершившемся дочернем процессе.

6. Сравните две программы

Вариант	**Что делает родитель**	**Zombie появляется**
zombie_bad.py	Не получает статус дочернего процесса	Да
zombie_good.py	Выполняет os.waitpid()	Нет

Контрольные вопросы:

1. **Почему в первой программе появился zombie?**
   Потому что дочерний процесс завершился, а родительский процесс не получил информацию о его завершении с помощью wait() или waitpid(). Поэтому запись о дочернем процессе временно осталась в таблице процессов.
2. **Какой процесс является родительским для zombie?**
   Родительским является тот процесс Python, который выполнил: os.fork(), Его PID можно увидеть в столбце PPID у zombie процесса.
3. **Что означает состояние Z?**
   Z означает **Zombie**. Процесс уже завершил выполнение, но запись о нём ещё хранится в таблице процессов.
4. **Почему команда kill -9 не решает проблему zombie?**
   Потому что zombie уже не выполняется. Его код уже завершён, поэтому «убивать» его нечего. Нужно, чтобы родительский процесс получил статус завершения дочернего процесса.
5. **Что делает os.waitpid()?**
   os.waitpid() позволяет родительскому процессу дождаться завершения конкретного дочернего процесса и получить его статус завершения.
   Например:
   os.waitpid(pid, 0)
   означает: дождаться завершения дочернего процесса с указанным PID.
6. **Почему во второй программе zombie не остаётся?**
   Потому что родитель вызывает: os.waitpid(pid, 0)
   и получает статус завершения дочернего процесса. После этого ядро удаляет запись дочернего процесса из таблицы процессов, поэтому состояние Zombie не сохраняется.

Основной вывод:
Zombie появляется, когда дочерний процесс уже завершился, но родитель ещё не получил его статус завершения. Вызов os.waitpid() позволяет родительскому процессу получить этот статус, после чего запись дочернего процесса удаляется из таблицы процессов

Этап 17. Исследование учебного процесса-сироты.
Процесс-сирота — это дочерний процесс, **родитель которого завершился раньше него**, а сам дочерний процесс продолжает работать.
В Linux такой процесс не уничтожается. Ядро назначает ему нового родителя. В учебной виртуальной машине таким новым родителем обычно становится системный процесс с PID 1 (systemd).

На этом этапе создайте процесс-сироту и проследите, как изменяется его PPID.

1. Создайте программу

Создайте файл:

nano orphan.py

Введите следующий код:

```
#!/usr/bin/env python3

import os
import time

# Создаём дочерний процесс
pid = os.fork()

if pid == 0:
    # Эта часть выполняется дочерним процессом

    print("Child PID:", os.getpid())

    # Показываем PID текущего родителя
    print("Initial PPID:", os.getppid())

    # Ждём, пока родительский процесс завершится
    time.sleep(5)

    # После завершения исходного родителя
    # проверяем PPID ещё раз
    print("New PPID:", os.getppid())

    # Оставляем дочерний процесс работать ещё 30 секунд,
    # чтобы его можно было исследовать командой ps
    time.sleep(30)

else:
    # Эта часть выполняется родительским процессом

    print("Parent PID:", os.getpid())
    print("Child PID:", pid)

    # Родитель работает только 2 секунды
    time.sleep(2)

    # После этого родительский процесс завершается
    print("Parent process finished.")
```

Сохраните файл.

2. Сделайте файл исполняемым
Выполните:
chmod +x orphan.py
Команда chmod +x добавляет право на выполнение файла как программы.

3. Запустите программу
Выполните:
./orphan.py

Примерный вывод:
Parent PID: 3120
Child PID: 3121
Child PID: 3121
Initial PPID: 3120

Parent process finished.

New PPID: 1

Обратите внимание на две строки:
Initial PPID: 3120
New PPID: 1
Сначала дочерний процесс имеет родителя с PID 3120.
После завершения родителя значение PPID изменяется.

4. Зафиксируйте PID дочернего процесса
Запишите значение:
Child PID
Например:
Child PID: 3121
Этот PID понадобится для дальнейшей проверки.

5. Исследуйте процесс после завершения родителя
Пока дочерний процесс ещё работает, выполните:
ps -o pid,ppid,stat,cmd -p 3121
Замените 3121 на PID вашего дочернего процесса.
Пример:
PID PPID STAT CMD
3121   1 S   python3 ./orphan.py

Поля означают:
- PID — идентификатор самого процесса;

- PPID — идентификатор родительского процесса;
- STAT — состояние процесса;
- CMD — выполняемая команда.

Зафиксируйте:

- PID;
- PPID;
- состояние процесса.

6. Определите, стал ли процесс сиротой

Сравните PPID до и после завершения родителя.

До:

PID = 3121

PPID = 3120

После:

PID = 3121

PPID = 1

Если исходный родитель завершился, а дочерний процесс продолжает работать и получил нового родителя, значит он стал **процессом-сиротой**.

7. Исследуйте дерево процессов

Выполните:

pstree -p

Найдите процесс:

python3

с нужным PID.

Определите, под каким системным процессом он теперь находится.

8. Дождитесь завершения процесса

Через 30 секунд дочерний процесс завершится самостоятельно.

Проверьте:

ps -p 3121

Если строка больше не выводится, процесс завершён.

Что происходит в программе

Логика программы:

Родительский процесс

↓

создаёт дочерний процесс

↓

оба процесса работают одновременно

↓

родитель завершается через 2 секунды

↓

дочерний процесс продолжает работать

↓
он становится процессом-сиротой
↓
Linux назначает ему нового родителя
↓
PPID изменяется

Сравнение zombie и orphan

Ситуация	Что произошло
Zombie	Дочерний процесс завершился раньше родителя
Orphan	Родительский процесс завершился раньше дочернего

У zombie: STAT = Z
У orphan отдельной метки O или Orphan в ps нет.
Его определяют по тому, что:
исходный родитель завершился
+
дочерний процесс продолжает работать
+
PPID изменился

Контрольные вопросы

1. **Что такое процесс-сирота?**
   Процесс-сирота — это дочерний процесс, родитель которого уже завершился, а сам дочерний процесс продолжает работать.
2. **Почему дочерний процесс продолжает работать после завершения родителя?**
   Потому что дочерний процесс — это самостоятельный процесс со своим PID и состоянием. Завершение родителя не означает автоматического завершения ребёнка.
3. **Что показывает PPID?**
   PPID — это идентификатор родительского процесса, то есть PID процесса, который создал данный процесс.
4. **Как изменился PPID дочернего процесса после завершения родителя?**
   Сначала PPID был равен PID исходного родителя. После завершения родителя он изменился на PID нового родителя, обычно:
   PPID = 1
   То есть процесс был переподчинён системному процессу systemd.
5. **Можно ли определить orphan-процесс по отдельному значению в поле STAT?**
   Нет. Для orphan-процесса нет отдельного состояния вроде O.

Его определяют по ситуации: исходный родитель завершился, а дочерний процесс продолжает работать и его PPID изменился.

6. **В чём главное отличие процесса-сироты от zombie-процесса?**
   Процесс-сирота **ещё работает**, просто его исходный родитель уже завершился.
   Zombie-процесс, наоборот, **уже завершился**, но его запись ещё остаётся в таблице процессов, потому что родитель не забрал статус завершения.

Коротко:

- Orphan → родитель умер, ребёнок ещё работает
- Zombie → ребёнок умер, запись о нём ещё осталась

Этап 18. Итоговый эксперимент с нагрузкой
Запустите CPU-нагрузку:
yes > /dev/null &
PID=$!
Получите сведения:
ps -o pid,ppid,stat,ni,%cpu,%mem,cmd -p $PID
Затем:
free -h
И:
uptime
Команда uptime показывает, в частности, значения load average.
Зафиксируйте:

- PID;
- PPID;
- состояние;
- Nice;
- %CPU;
- %MEM;
- load average.

Откройте:
top
Найдите процесс.
После этого завершите:
kill $PID
Через некоторое время снова:
uptime
Сравните нагрузку до и после завершения процесса.

Этап 19. Очистка учебного стенда
Убедитесь, что экспериментальные процессы завершены:

jobs
Если фоновые задачи ещё работают, получите их PID и завершите:
kill PID
Удалите учебный файлы:
rm zombie_bad.py
rm zombie_good.py
rm orphan.py

ЧАСТЬ 2. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ ДЛЯ WINDOWS SERVER (Анализ многопоточности и служб)

Этап 1: Исследование потоков внутри процессов через Диспетчер задач

1. Откройте **Диспетчер задач** (Task Manager) ➔ перейдите на вкладку **«Подробности»** (Details).
2. Нажмите правой кнопкой мыши по заголовкам таблиц (например, где написано *Имя*, *PID*) ➔ выберите пункт **«Выбрать столбцы»** (Select columns).
3. Найдите и активируйте столбец **«Потоки»** (Threads). Нажмите ОК.
4. Отсортируйте процессы по количеству потоков. Найдите системный процесс **lsass.exe** (Local Security Authority Subsystem Service) или **services.exe**. Зафиксируйте в отчете его PID и количество выполняемых внутри него потоков.

Этап 2: Мониторинг сетевых соединений процесса через Resource Monitor

Каждый запущенный процесс имеет право открывать файлы и сетевые порты.

1. Нажмите Win + R, введите resmon для запуска **Монитора ресурсов** (Resource Monitor).
2. Перейдите на вкладку **«Сеть»** (Network).
3. Разверните вкладку **«Сетевые подключения»** (Network Connections).

4. Найдите процесс веб-сервера (если установлен IIS, это будет w3wp.exe) или службы DNS/Active Directory. Если сервер «чистый», найдите процесс svchost.exe.
5. Запишите в отчет: имя процесса, его PID, локальный порт, который он слушает (Local Port), и адрес, с которым он связан.

Этап 3: Инвентаризация и аудит системных служб через GUI (Графический интерфейс)

В Windows Server управление фоновыми процессами (службами) разделено между классической оснасткой «Службы» и Диспетчером задач. Администратор должен уметь быстро находить взаимосвязи между запущенной службой и реальным процессом в памяти.

1. **Поиск и анализ конкретной службы:**
    - Нажмите комбинацию клавиш Win + R, введите **services.msc** и нажмите Enter. Откроется графическая оснастка **«Службы»** (Services).
    - Найдите в алфавитном списке службу **«Диспетчер печати»** (Print Spooler).
    - Нажмите по ней правой кнопкой мыши ➔ **Свойства** (Properties).
    - Зафиксируйте в отчете: *Имя службы (Service name)*, *Имя исполняемого файла (Path to executable)* и текущий *Тип запуска (Startup type)*.
2. **Связывание службы с процессом в памяти (PID):**
    - Перейдите на вкладку **«Состояние»** в этом же окне свойств и убедитесь, что служба запущена (Running).
    - Откройте **Диспетчер задач** (Ctrl + Shift + Esc) ➔ перейдите на вкладку **«Службы»** (Services).
    - Найдите в списке службу по её короткому имени Spooler.
    - Посмотрите в столбец **PID** (Идентификатор процесса). Запишите это число.
    - Нажмите по службе правой кнопкой мыши и выберите пункт **«Перейти к процессам»** (Go to details). Диспетчер задач

автоматически перекинет вас на вкладку «Подробности» и подсветит исполняемый .exe файл, внутри которого крутится эта служба. Запишите имя этого файла.

3. **Поиск скрытых проблем (Остановленные автоматические службы):**
    - Вернитесь в окно services.msc.
    - Нажмите на заголовок столбца **«Состояние»** (Status), чтобы отсортировать службы.
    - Найдите глазами службы, у которых в колонке **«Тип запуска»** стоит *«Автоматически»* (Automatic), но при этом колонка **«Состояние»** (Status) пустая (то есть служба остановлена).
    - *Выпишите в отчет имя любой одной такой службы, если она есть в системе.*

Шпаргалка-памятка:

- **В Linux:** Помните, что zombie-процесс — это не «живой» процесс, он не потребляет процессорное время (CPU) или оперативную память, а является лишь мертвой записью в таблице процессов, которую родитель забыл считать с помощью системного вызова wait(). Убить зомби командой kill -9 невозможно, так как он уже мертв. Нужно перезапускать его родительский процесс.
- **В Windows Server:** Столбец «Потоки» в Диспетчере задач наглядно демонстрирует, почему один процесс (например, браузер или СУБД) может одновременно выполнять много задач (скачивать файл, отрисовывать страницу, слушать сеть) — под каждую подзадачу выделяется свой легковесный поток внутри общего адресного пространства процесса
- в Windows это устроено не так явно, как в Linux.
- В Window аналог процесса-сироты есть. Дочерний процесс не обязан завершаться вместе с родительским. Родитель может завершиться, а дочерний продолжит работать. Однако Windows

не «переподчиняет» такой процесс PID 1, как Linux. У процесса сохраняется информация о своём родителе, но сам родитель уже может не существовать. Поэтому отдельного статуса orphan в Task Manager нет.

- С zombie ситуация другая. Классического Linux-zombie в Windows в обычном смысле нет. В Linux завершившийся дочерний процесс может оставаться в таблице процессов до тех пор, пока родитель не заберёт его статус через wait(). В Windows механизм завершения процессов и объектов ядра другой: после завершения процесса его ресурсы освобождаются, а объект процесса может некоторое время существовать, пока на него остаются открытые дескрипторы. Но это не тот же самый Zombie со статусом Z.